To define an event for these hooks, use the block syntax within a `Rails::Application`, `Rails::Railtie` or `Rails::Engine` subclass:

```
module YourApp
  class Application < Rails::Application
    config.before_initialize do
      # initialization code goes here
    end
  end
end
```

Alternatively, you can also do it through the `config` method on the `Rails.application` object:

```
Rails.application.config.before_initialize do
  # initialization code goes here
end
```

Some parts of your application, notably routing, are not yet set up at the point where the `after_initialize` block is called.

6.1 `Rails::Railtie#initializer`

Rails has several initializers that run on startup that are all defined by using the `initializer` method from `Rails::Railtie`. Here's an example of the `set_helpers_path` initializer from Action Controller:

```
initializer "action_controller.set_helpers_path" do |app|
  ActionController::Helpers.helpers_path = app.helpers_paths
end
```

The `initializer` method takes three arguments with the first being the name for the initializer and the second being an options hash (not shown here) and the third being a block. The `:before` key in the options hash can be specified to specify which initializer this new initializer must run before, and the `:after` key will specify which initializer to run this initializer *after*.

Initializers defined using the `initializer` method will be run in the order they are defined in, with the exception of ones that use the `:before` or `:after` methods.

You may put your initializer before or after any other initializer in the chain, as long as it is logical. Say you have 4 initializers called "one" through "four" (defined in that order) and you define "four" to go *before* "four" but *after* "three", that just isn't logical and Rails will not be able to determine your initializer order.

The block argument of the `initializer` method is the instance of the application itself, and so we can access the configuration on it by using the `config` method as done in the example.

Because `Rails::Application` inherits from `Rails::Railtie` (indirectly), you can use the `initializer` method in `config/application.rb` to define initializers for the application.

6.2 Initializers

Below is a comprehensive list of all the initializers found in Rails in the order that they are defined (and therefore run in, unless otherwise stated).

- `load_environment_hook` Serves as a placeholder so that `:load_environment_config`can be defined to run before it.
- `load_active_support` Requires `active_support/dependencies` which sets up the basis for Active Support. Optionally requires `active_support/all` if `config.active_support.bare` is un-truthful, which is the default.
- `initialize_logger` Initializes the logger (an `ActiveSupport::Logger` object) for the application and makes it accessible at `Rails.logger`, provided that no initializer inserted before this point has defined `Rails.logger`.
- `initialize_cache` If `Rails.cache` isn't set yet, initializes the cache by referencing the value in `config.cache_store` and stores the outcome as `Rails.cache`. If this object responds to